Sandvik Mining and Construction Australia

Tunnel Convergence Sensing System

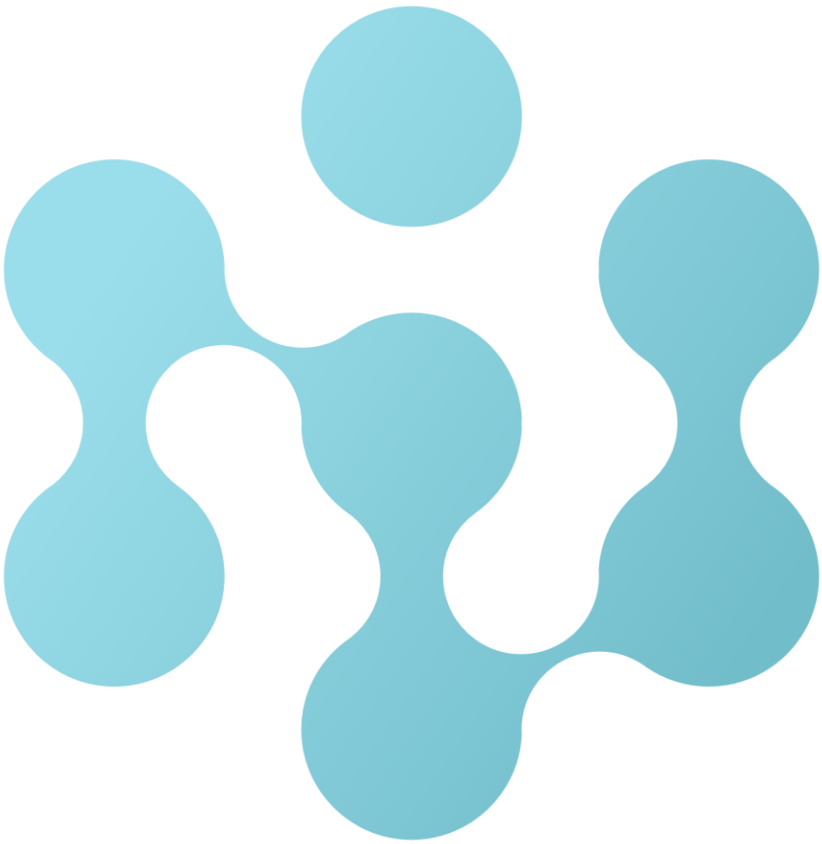
GATT Profiles

Document Number: D0010553

Revision: 1.06

Author: D. Berton

Date: 28 April 2026



Copyright © 2026 Ingenuity Design Group Pty Ltd
All Rights Reserved

Suite 2.2 56 Delhi Road
North Ryde NSW 2113

Email: enquiries@ingenuity-design.com.au
Phone: +61 1300 611 388

Document Control

Document Title	Tunnel Convergence Sensing System GATT Profiles
Document Number	D0010553
Revision	1.06
Date	28 April 2026
Author	D. Berton
Reviewer	D. Knight
Approver	D. Knight
Template No. (Rev)	D0002554 (1.06)

Document History

Revision	Description of Changes	Date	Revised	Approved
1.00	Initial Release	14 Jul 2021	DK	JM
1.01	Added timestamp to configuration	27 Jul 2021	DK	JM
1.02	Updated to match released devices	04 Mar 2022	DK	JM
1.03	Add more configurations and commands	6 Dec 2022	DB	AI
1.04	Update GW config properties	6 Mar 2023	DB	AI
1.05	Add Hydra and update prev. device naming, add Wi-Fi mac retrieval	23 Oct 2025	DB	DK
1.06	Hydra: Remove unused Amplitude Threshold config, remove unused config ID 5, add Clear Samples command, increase size of interrogator init strings and make them "long configuration"	28 Apr 2026	DB	DK

Contents

1 Introduction 7
1.1 General 7
1.2 Definitions 7
1.2.1 Acronyms 7
1.2.2 Terms 7
1.3 Reference Documents 8
2 System 9
2.1 Base UUID 9
2.2 Advertisements 9
2.2.1 Manufacturer Specific Data 9
2.3 Read/Write operation types 10
2.4 Long Configuration values 10
3 Cyclops 11
3.1 Configuration Service (UUID: 0001) 13
3.1.1 Set Configuration Characteristic (UUID: 0011) 14
3.1.2 Get Configuration Characteristic (UUID: 0012) 14
3.1.3 Configuration Characteristic (UUID:0013) 14
3.1.4 Wi-Fi configuration 14
3.2 Wi-Fi configuration 14
3.2.1 Unencrypted Personal Networks 15
3.2.2 Encrypted Personal Networks 15
3.2.3 Enterprise Networks - PEAP/MSCHAPV2 15
3.2.4 Enterprise Networks - TLS 15
3.2.5 Enterprise Networks - TTLS 16
3.3 Measurement Service (UUID: 0002) 16
3.3.1 Retrieve Measurements Characteristic (UUID: 00021) 16
3.3.2 Measurement Meta Characteristic (UUID: 0022) 17
3.3.3 Measurement Characteristic (UUID: 0023) 17
3.4 Status Service (UUID: 0003) 18
3.4.1 Device Status Characteristic (UUID:0031) 18
3.4.2 Button Characteristic (UUID: 0032) 19
3.4.3 Command Characteristic (UUID: 0033) 19

3.5 Device Information Service (UUID: 180A) 21
3.5.1 Manufacturer Name String Characteristic (UUID: 2A29) 21
3.5.2 Model Number String Characteristic (UUID: 2A24) 22
3.5.3 Firmware Revision String Characteristic (UUID: 2A26) 22
3.5.4 Software Revision String Characteristic (UUID: 2A28) 22
3.5.5 Hardware Revision String Characteristic (UUID: 2A27) 22
3.5.6 Serial Number String Characteristic (UUID: 2A25) 22
3.6 Generic Access Service (UUID: 1800) 22
3.6.1 Device Name Characteristic (UUID: 2A00) 22
3.6.2 Appearance Characteristic (UUID: 2A01) 22
3.7 Silicon Labs OTA Service (UUID: 1D14D6EE-FD63-4FA1-BFA4-8F47B42119F0) 22
4 Hydra 23
Configuration Service (UUID: 0001) 25
4.1.1 Set Configuration Characteristic (UUID: 0011) 26
4.1.2 Get Configuration Characteristic (UUID: 0012) 26
4.1.3 Configuration Characteristic (UUID:0013) 27
4.1.4 Wi-Fi configuration 27
4.2 Wi-Fi configuration 27
4.2.1 Unencrypted Personal Networks 27
4.2.2 Encrypted Personal Networks 27
4.2.3 Enterprise Networks - PEAP/MSCHAPV2 27
4.2.4 Enterprise Networks - TLS 28
4.2.5 Enterprise Networks - TTLS 28
4.3 Measurement Service (UUID: 0002) 28
4.3.1 Retrieve Measurements Characteristic (UUID: 00021) 28
4.3.2 Measurement Meta Characteristic (UUID: 0022) 30
4.3.3 Measurement Characteristic (UUID: 0023) 30
Status Service (UUID: 0003) 30
4.3.4 Button Characteristic (UUID: 0032) 30
4.3.5 Device Status Characteristic (UUID:0031) 30
4.3.6 Command Characteristic (UUID: 0033) 31
4.4 Device Information Service (UUID: 180A) 32
4.4.1 Manufacturer Name String Characteristic (UUID: 2A29) 32
4.4.2 Model Number String Characteristic (UUID: 2A24) 32

4.4.3 Firmware Revision String Characteristic (UUID: 2A26) 32
4.4.4 Software Revision String Characteristic (UUID: 2A28) 32
4.4.5 Hardware Revision String Characteristic (UUID: 2A27) 33
4.4.6 Serial Number String Characteristic (UUID: 2A25) 33
4.5 Generic Access Service (UUID: 1800) 33
4.5.1 Device Name Characteristic (UUID: 2A00) 33
4.5.2 Appearance Characteristic (UUID: 2A01) 33
5 Professor 34
5.1 Configuration Service (UUID: 0101) 35
5.1.1 Set Configuration Characteristic (UUID: 0111) 36
5.1.2 Get Configuration Characteristic (UUID: 0012) 36
5.1.3 Configuration Characteristic (UUID:0013) 37
5.2 Shutdown Timeout 37
5.3 Wi-Fi configuration 37
5.3.1 Unencrypted Personal Networks 37
5.3.2 Encrypted Personal Networks 37
5.3.3 Enterprise Networks - PEAP/MSCHAPV2 38
5.3.4 Enterprise Networks - TLS 38
5.3.5 Enterprise Networks - TTLS 39
5.3.6 Supplementary information 39
5.4 Status Service (UUID: 0102) 40
5.4.1 Device Status Characteristic (UUID:0121) 40
5.4.2 Command Characteristic (UUID: 0122) 40
5.5 Device Information Service (UUID: 180A) 41
5.5.1 Manufacturer Name String Characteristic (UUID: 2A29) 41
5.5.2 Model Number String Characteristic (UUID: 2A24) 41
5.5.3 Firmware Revision String Characteristic (UUID: 2A26) 41
5.5.4 Software Revision String Characteristic (UUID: 2A28) 41
5.5.5 Hardware Revision String Characteristic (UUID: 2A27) 41
5.5.6 Serial Number String Characteristic (UUID: 2A25) 41
5.6 Generic Access Service (UUID: 1800) 42
5.6.1 Device Name Characteristic (UUID: 2A00) 42
5.6.2 Appearance Characteristic (UUID: 2A01) 42
Appendix A. Measurement Encoding scheme 43

A.1 Hydra Measurement 43
A.2 Transport Layer 43
A.3 Application layer 44
A.4 Hydra Measurement (Packet Type: 1) 44

1 Introduction

1.1 General

This document describes the GATT Profiles for the Tunnel Convergence Sensing System developed by Ingenuity Design Group Pty Ltd (Ingenuity) on behalf of Sandvik Mining and Construction Australia. These GATT profiles describe the protocol used for BLE communications with the Sensor and Professor devices.

1.2 Definitions

1.2.1 Acronyms

The following abbreviations are used in this document:

AP	Access Point
API	Application Programming Interface
BLE	Bluetooth Low Energy
CA	Certificate Authority
FIFO	First In, First Out
GATT	General ATTribute
LED	Light Emitting Diode
MTU	Maximum Transmission Unit
NA	Not Applicable
OTA	Over-The-Air [Firmware Upgrade]
PSK	Pre-Shared Key
SIG	Special Interest Group
SSID	Simple Service Identifier
TBA	To Be Advised
TBC	To Be Confirmed
TCP	Transmission Control Protocol
TCSS	Tunnel Convergence Sensing System
URL	Uniform Resource Locater
UUID	Universally Unique ID

1.2.2 Terms

The following terms are used in this document:

Shall, Must	Indicates a mandatory requirement.
Should	Indicates a recommendation.
Will	Indicates a non-mandatory provision with a declaration of intent.
May	Indicates a permission.
Can	Indicates a possibility or a capability
Note	Used to designate additional information intended to provide guidance, understanding and/or clarification

1.3 Reference Documents

The following documents are referenced in this report.

Ref.	Title	Doc No.	Author	Rev	Date
D1	Sandvik TCSS Product Specification	D0010341	SN	1.01	26 Jul 2021

2 System

2.1 Base UUID

The TCSS uses the custom base UUID of bdeeXXXX-7e07-4e68-85f1-d4d87a014940

2.2 Advertisements

All BLE devices advertise the following information:

AD Type	Contents	Notes
Local Name	"SVK" followed by the zero-padded serial number	Eg, SVK000004451, can be set by user on sensor only.
Manufacturer Specific Data	See Below	Eg, FF-FF-E4-03-6D-F5-00-06-00-00

Advertisement interval is configurable, and defaults to 500ms.

2.2.1 Manufacturer Specific Data

The Manufacturer ID is currently 0xFFFF (Reserved ID for testing/development). This can be replaced with an ID assigned by the Bluetooth SIG when required.

The following data is advertised in the Manufacturer Specific Data

Description	*Type*	*Possible Values*
Magic Value	uint32	0xF56D03E4
Device Type	uint8	0: Cyclops, v1 1: Professor, v1 2: Hydra, v1 Other values are reserved for future use
Status Flags	uint8	Bit 0: Low Batt Bit 1: Priority Data Available Bit 2: Data Available Bit 3: Buffer > 75% full Bit 4: Reserved for Future Use Bit 5: BLE Upload Disabled Bit 6: Error information available Bit 7: Fatal Error (indicating not to connect)
Age of oldest sample*	uint16	0 ~ 65535 min (0 ~ 45.5 days)

* To conserve battery, **Age of oldest sample** is not updated every minute, it is only updated after samples are read/written to disk, or after a BLE connection completes.

Note : Professor Manufacturing data is a constant value of 0xE4, 0x03, 0x6D, 0xF5, 0x01, 0x00, 0xFF, 0xFF.

2.3 Read/Write operation types

All read and write operations shall be Read/Write Requests. All asynchronous data transfers will be sent as Notifications.

2.4 Long Configuration values

The maximum length of a regular configuration values is a maximum of 200 bytes long. Some configuration values may need to be up to 50kB long. To get around the 200-byte limit, the configuration must be split into blocks and written sequentially instead.

The blocks shall be written to the "configuration value" in the "Set Configuration" characteristic. Each configuration that uses this block transfer protocol is indicated in the configuration tables

Each block must have a 2-byte header prepended to it; this is for validation during reassembly. The first header byte contains the index of the current block (i.e for the first block, index =0) The second byte contains total number of blocks to be sent.

Even if a "Long Configuration" value fits within one block, it should still have the header prepended with header bytes set to 0x0,0x01. Setting a "Long Configuration" value to the empty string can be done by sending only the 0x0,0x01 header.

Each block (including header) must fit within the 193-byte maximum of the Configuration service, so block data must be between 0 bytes and 191 bytes long.

Each block except the final block must have a payload size that is modulo 4, thus further limiting the maximum block payload size to 188 bytes.

The blocks are expected to be sent in order with no missing values. If an error is detected, no response will be returned.

Once the final block has been written to the "Set Configuration" characteristic and validated successfully, a zero-length configuration packet will be sent back via the "Configuration characteristic" using the same protocol.

An example is presented below where the block payload size is chosen to be a maximum of four bytes (for simplicity).

The desired configuration value is the string 'abcdefghij' (not including quotes). Sending this with 4 bytes/block, this requires three blocks to be sent as the "configuration value" in the "Set Configuration" characteristic:

0x00 ,0x03, 0x61, 0x62, 0x63, 0x64 # payload length must be a multiple of 4 (max size: 188)
0x01 ,0x03, 0x65, 0x66, 0x67, 0x68 # payload length must be a multiple of 4 (max size: 188)
0x02 ,0x03, 0x69, 0x6A # last block payload length can have any length up to 191 bytes

3 Cyclops

Configuration Service — UUID: bdee0001-7e07-4e68-85f1-d4d87a014940

Set Configuration (Size: 8+ bytes)		
Properties: W		UUID 0011
Type	Attribute	Possible Values
uint16	Configuration Item Id to Set	0 ~ 255
uint8	Length of Value	0 ~ 200
uint32	Update time	1/1/1970 ~ 7/2/2038 (s) (or 0 for now)
byte[0~200]	Configuration Value	

Get Configuration (Size: 2 bytes)		
Properties: W		UUID 0012
Type	Attribute	Possible Values
uint16	Configuration Item Id to Get	0 ~ 255

Configuration (Size: 8+ bytes)		
Properties: R/Notify		UUID 0013
Type	Attribute	Possible Values
uint16	Configuration Item Id	0 ~ 255
uint8	Length of Value	0 ~ 200
uint32	Last Update Time	1/1/1970 ~ 19/1/2038 (s)
byte[0~200]	Configuration Value	

Measurement Service — UUID: bdee0002-7e07-4e68-85f1-d4d87a014940

Retrieve Measurements (Size: 2+ bytes)		
Properties: W		UUID 0021
Type	Attribute	Possible Values
uint8	Command	0 ~ 3
uint8	Number of Packets	0 ~ 16
uint8[]	Packet Ids to Acknowledge	

Measurement Meta (Size: 4 bytes)		
Properties: R/Notify		UUID 0022
Type	Attribute	Possible Values
uint16	Number of Measurements	0 ~ 1000
uint16	Number of Non-Uploaded Measurements	0 ~ 1000

Measurement (Size: 16+ bytes)		
Properties: Notify		UUID 0023
Type	Attribute	Possible Values
uint8	Packet Id	0 ~ 255
uint8	Sample Count	0 ~ 255
uint32	Sample Id [0]	0 ~ 4294967296
uint32	Timestamp[0]	1/1/1970 ~ 7/2/2106 (s)
uint16	Distance Measurement[0]	0 ~ 65535mm
uint16	Measurement Quality	0-65535
uint8	Measurement Duration	0-256, in units of 64ms
uint32	Gravity xyz[0]	3x 10 bit (±2g scaled)
	...	
uint32	Sample Id [n]	0 ~ 4294967296
uint32	Timestamp[n]	1/1/1970 ~ 7/2/2038 (s)
uint16	Distance Measurement[n]	0 ~ 65535mm
uint16	Measurement Quality	0-65535
uint8	Measurement Duration	0-256, in units of 64ms
uint32	Gravity xyz[n]	3x 10 bit (±2g scaled)

Status Service — UUID: bdee0003-7e07-4e68-85f1-d4d87a014940

Device Status (Size: 20 bytes)		
Properties: R		UUID 0031
Type	Attribute	Possible Values
uint8	Status	0 ~ 2
uint32	Faults	Flags
uint16	Battery Voltage	0 ~ 5500 mV
uint8	Battery Status	0 ~ 2
uint32	Current Timestamp	1/1/1970 ~ 19/1/2038 (s)
uint16	Last Distance Measurement	0 ~ 65535mm+
uint32	Last Gravity xyz	3x 10 bit (±2g scaled)
int8	Temperature	-128 ~ 127 °C
uint16	Last Sensor Quality	0-65535

Button (Size: 1 byte)		
Properties: R/W/Notify		UUID 0032
Type	Attribute	Possible Values
uint8	Button Pressed	0-1

Command (Size: 6+ bytes)		
Properties: W/Notify		UUID 0033
Type	Attribute	Possible Values
uint8	Command	0 ~ 11, 254, 255
uint32	Parameter	
int8	Result Code	
uint8	Result Length	0 ~ 240
uint8[]	Result Data	

Device Info UUID: 180A

Manufacturer Name String (Size: 7 bytes)		
Properties: R		UUID: 2A29
Type	Attribute	Possible Values
utf8	Manufacturer Name	Sandvik

Model Number String (Size: 5 bytes)		
Properties: R		UUID: 2A24
Type	Attribute	Possible Values
utf8	Model Number	XX.XX

Firmware Revision String (Size: 5 bytes)		
Properties: R		UUID: 2A26
Type	Attribute	Possible Values
utf8	Firmware Revision	XX.XX

Software Revision String (Size: 5 bytes)		
Properties: R		UUID: 2A28
Type	Attribute	Possible Values
utf8	Software Revision	XX.XX

Hardware Revision String (Size: 5 bytes)		
Properties: R		UUID: 2A27
Type	Attribute	Possible Values
utf8	Hardware Revision	XX.XX

Serial Number String (Size: 9 bytes)		
Properties: R		UUID: 2A25
Type	Attribute	Possible Values
utf8	Serial Number	000000001 ~ 999999999

Generic Access UUID: 1800

Device Name (Size: Up to 11 bytes)		
Properties: R/W		UUID: 2A00
Type	Attribute	Possible Values
utf8	Device Name	

Appearance (Size: 2 byte)		
Properties: R		UUID: 2A01
Type	Attribute	Possible Values
uint16	Appearance	0

Silicon Labs OTA 1D14D6EE-FD63-4FA1-BFA4-

OTA Control (Size: 1 byte)		
Properties: W		F7BF3564-FB6D-4E53-88A4-5E37E0326063
Type	Attribute	Possible Values
uint8	OTA Control	0, 1, 3

OTA Data (Size: 20 bytes | 244 bytes?)		
Properties: W		984227F3-34FC-4045-A5D0-2C581F81A153
Type	Attribute	Possible Values
uint8	OTA Data[]	

3.1 Configuration Service (UUID: 0001)

The Configuration Service allows getting and setting of configurations in the Cyclops. The following Configuration Items are available:

ID	Type	Description	Default
1	uint8	BLE Upload Enable – 0: Disabled, Otherwise Enabled	enabled
2	uint8	Wi-Fi Upload Enable – 0: Disabled, Otherwise Enabled	enabled
3	String	Wi-Fi SSID – max 31 characters	wifi
4*	String	Wi-Fi PSK – max 31 characters	wifiwifi
5	uint16	Measurement Period – in minutes (0-65535)	15 minutes
6	uint16	Wi-Fi Upload Period – in minutes (0-65535)	60 minutes
7+	String	Server URL – max 47 characters	
8*+	String	API Key – max 47 characters	
9	uint16	Distance Priority Threshold – in mm (0-65535)	50mm
10	uint16	Orientation Priority Threshold – in units of 0.0039G (0-1024)	64 (0.25G)
11	uint16	Laser Repeat Count (0-50)	5
12	uint16	Laser Repeat Interval in seconds (0-3600)	60s
13	uint16	Status LED Flash Interval in minutes (0-65535)	0 (disabled)
14	uint16	Advertising Interval – in ms (0-65535)	500ms
15	uint16	Laser Timeout – in ms (0-10000)	6000ms
16	uint8	Wi-Fi network key management protocol [0 = None 1 = WEP, 2 = WPA, 3 = WPA2, 4 = EAP]	3
17	uint8	WPA-EAP Protocol [0= TLS, 1 = PEAP, 2 = TTLS]	0
18*	String	Wi-Fi EAP Anonymous ID – max 31 characters	
19*	String	Wi-Fi EAP ID – max 31 characters	
20	String	Wi-Fi NTP Address – max 43 characters	pool.ntp.org
21	uint8	Wi-Fi NTP Enabled – 0: Disabled, Otherwise Enabled	enabled
129+	uint32	Serial Number (0-4294967295)	0
130	String	BLE advertising name - max 13 characters (set first character as space (0x20) for default value)	SVK + serial number
209*#	uint8[]	AtPKI encoded binary blob containing RADIUS Server CA Certificate(s) for WPA-EAP	
210*#	uint8[]	AtPKI encoded binary blob containing Client Private key(s) for WPA-EAP	
211*#	uint8[]	AtPKI encoded binary blob containing Client Certificate(s) for WPA-EAP	

* These items are write-only and cannot be read.

+ These items are not reset during a Factory Reset

Items are considered "Long Configuration" values and must be read and written using the "Long Configuration" protocol, described in section 2.4.

Note: AtPKI is a container for multiple certificate and key files. Its format is described here

The Configuration Service consists of the following characteristics:

3.1.1 Set Configuration Characteristic (UUID: 0011)

Write the configuration item id, the length of the value, the configuration timestamp, and the new value to the characteristic. The data is validated and applied immediately.

For data to be accepted,

- The provided configuration id must exist
- The provided length must match the provided data and the expected length
- The provided value must be within the permitted values
- The maximum length of configuration value is 200 bytes
- The provided timestamp must be later than the timestamp of the current value, or 0 (which signifies 'now', and will be replaced with the current timestamp)

If the new values are acceptable, a notification is sent to the Configuration Characteristic (UUID: 0013). If a write-only or Long-Configuration value is updated, a notification is still sent, but the value is not included in the notification.

The configuration values are stored to flash once the Bluetooth connection is disconnected.

3.1.2 Get Configuration Characteristic (UUID: 0012)

Write the desired configuration id to the characteristic. If the id exists, the current value of the configuration will be sent as a notification to the Configuration Characteristic (UUID: 0013). If a write-only value is requested, a notification is still sent, but the value is not included in the notification.

For Configuration Items set to their default value, the Last Updated Timestamp will be 0.

3.1.3 Configuration Characteristic (UUID:0013)

This characteristic holds the value of the configuration item most recently requested or updated.

3.1.4 Wi-Fi configuration

3.2 Wi-Fi configuration

The Cyclops uses ESP-AT firmware on a ESP32-C3 module to connect to wireless networks.

Once the Wi-Fi configuration has been changed, the new configuration will be updated after the next time the Cyclops periodically connects to a Wi-Fi network. The 'Connect to configured Wi-Fi AP' command can be used to verify the network configuration is valid.

The following subsections describe how to configure various types of supported Wi-Fi networks.

3.2.1 Unencrypted Personal Networks

Set the following configuration values:

- `WifiSSID` (**3**) to your SSID
- `WifiKeyMgmtProto` (**16**) to 0 (None)

3.2.2 Encrypted Personal Networks

Set the following configuration values:

- `WifiSSID` (**3**) to your SSID
- `WifiPassword` (**4**) set to your Wi-Fi networks PSK
- `WifiKeyMgmtProto` (**16**) to one of 1 (WEP), 2 (WPA), or 3 (WPA2)

3.2.3 Enterprise Networks - PEAP/MSCHAPV2

Set the following configuration values:

- `WifiSSID` (**3**) to your SSID
- `WifiPassword` (**4**) to the RADIUS server password
- `WifiKeyMgmtProto` (**16**) to 4 (WPA-EAP)
- `WifiEapProto` (**17**) to 1 (PEAP)
- `WifiEapAnonIdentity` (**18**) to the RADIUS server anonymous identity
- `WifiEapIdentity` (**19**) to the RADIUS server identity

Optionally the `WifiEapServerCAFile` (**209**) config value can be set to contain the RADIUS serv CA certificate to authenticate the server. The certificate should be encoded using the AtPKI file format and have its 'item type' = 'ca' and PKI index =0.

3.2.4 Enterprise Networks - TLS

Set the following configuration values:

- `WifiSSID` (**3**) to your SSID
- `WifiKeyMgmtProto` (**16**) to 4 (WPA-EAP)
- `WifiEapProto` (**17**) to 0 (TLS)
- `WifiEapAnonIdentity` (**18**) to the RADIUS server anonymous identity
- `WifiEapIdentity` (**19**) to the RADIUS server identity

- `WifiEapServerCAFile` (**209**) to an AtPKI file containing the server Certification Authority (CA) certificate. ('item type' = 'ca')
- `WifiEapClientPrivKeyFile` (**210**) to an AtPKI file containing the client private key ('item type' = 'key')
- `WifiEapClientCertFile` (**211**) to an AtPKI file containing the client certificate file ('item type' = 'cert')

The private key must be unencrypted for the connection to succeed.

3.2.5 Enterprise Networks - TTLS

Set the following configuration values:

- `WifiSSID` (**3**) to your SSID
- `WifiPassword` (**4**) to the RADIUS server password
- `WifiKeyMgmtProto` (**16**) to 4 (WPA-EAP)
- `WifiEapProto` (**17**) to 2 (TTLS)
- `WifiEapAnonIdentity` (**18**) to the RADIUS server anonymous identity
- `WifiEapIdentity` (**19**) to the RADIUS server identity
- `WifiEapServerCAFile` (**209**) to an AtPKI file containing the server Certification Authority (CA) certificate. ('item type' = 'ca')

3.3 Measurement Service (UUID: 0002)

The Measurement Service allows retrieving and acking of sets of measurements taken by the Cyclops.

The Measurement Service consists of the following characteristics:

3.3.1 Retrieve Measurements Characteristic (UUID: 00021)

Write commands to this characteristic to retrieve or acknowledge measurements from the Cyclops.

The commands available are:

Command	Description	Required Fields
0	Optionally ack listed packets, then Send **New** Samples	Command, Number of Packets. Packet Ids optional.
1	Optionally ack listed packets, then Send **All** Samples	Command, Number of Packets. Packet Ids optional.
2	Ack listed packets	Command, Number of Packets, and Packet Ids.
3	[Deprecated] Ack listed packets and continue sending packets.	Command, Number of Packets, and Packet Ids.

These commands are optimised for 2 main scenarios:

3.3.1.1 *Professor Download*

This scenario requires efficient retrieval of new samples only.

In this scenario, write command 0, and the number of packets to send. Up to that number of notifications will be sent to the Measurement Characteristic (UUID: 0023). To Acknowledge the packets, send command 0, the number of packets to send, and the ids of the packets sent previously.

This process can be repeated until there are no new samples transferred in response to the command. At this point, all samples stored in the Cyclops are acked.

The number of packets which can be requested is limited while using this command. A maximum of 16 packets can be pending acknowledgement at any time – requesting more than this will return empty packets until one or more pending packets are acknowledged.

3.3.1.2 *App Local History*

This scenario involves a user requesting the history of a specific Cyclops using an app via a local Bluetooth connection.

In this scenario, write command 1, and the number of packets to send. The packets will be sent as a series of notifications to the Measurement Characteristic (UUID: 0023). This can be repeated as required.

The packets sent in this scenario will not be able to be acknowledged, however, if previous packets are pending acknowledgment, providing their packet ids with this command will ack them.

In both scenarios, oldest samples are sent first, and as samples are sent, internal indexes keep track of which samples have been uploaded, and which have not. Once the most recent sample is uploaded, no further samples will be sent. Disconnecting from the Cyclops will reset the indexes and allow the same samples to be sent again.

3.3.2 Measurement Meta Characteristic (UUID: 0022)

This characteristic holds the current number of acknowledged and unacknowledged samples. This data can be read, or when the count changes, a notification will be sent.

During periods where this value changes rapidly, such as during acknowledgment of many packets, notifications will be paused until the value has not changed for 500ms.

3.3.3 Measurement Characteristic (UUID: 0023)

This characteristic is used to send notifications containing packets as requested using the Retrieve Measurements Characteristic (UUID: 0021). Subscribe to this characteristic before sending commands to ensure no data is missed.

Notifications sent to this characteristic will contain as many samples as will fit in the MTU, up to a maximum of 13.

Notifications sent will consist of a packet header, containing the Packet Id and Sample Count, followed by a set of samples. Each sample consists of the Sample Id (unique per-sensor) and timestamp, as well as the measured distance, quality of the measurement, time taken for the measurement to be taken (in units of 64ms), and an x/y/z acceleration measurement for determining orientation.

The orientation consists of 3x 10-bit values, packed into 32 bits as follows:

Bits	Type	Description
0 ~ 9	int10	Gravity X, +/- 2G
10 ~ 19	int10	Gravity Y, +/- 2G
20 ~ 29	int10	Gravity Z, +/- 2G
30		Reserved for future use
31		High Priority Measurement

The gravity values can be converted to multiples of G by using the following algorithm (assuming that the 32-bit orientation data is stored in a variable called gravity, and the outputs are stored in xOut, yOut, and zOut):

```
//Extract the 3x 10-bit values
int32_t x = gravity & 0x3FF;
int32_t y = (gravity >> 10) & 0x3FF;
int32_t z = (gravity >> 20) & 0x3FF;

//Sign-extend the variables to correctly handle negative values
if (x & 0x200)
  x |= 0xFFFFFC00;

if (y & 0x200)
  y |= 0xFFFFFC00;

if (z & 0x200)
  z |= 0xFFFFFC00;

//Scale the values to G
float xOut = x * 2.0 / 512;
float yOut = y * 2.0 / 512;
float zOut = z * 2.0 / 512;
```

3.4 Status Service (UUID: 0003)

The Status Service consists of the following characteristics

3.4.1 Device Status Characteristic (UUID:0031)

This characteristic holds the status of the Cyclops. It consists of the following values:

Attribute	Description

Status	0: Good 1: Non-Fatal Error(s) Present 2: Fatal Errors Present
Errors	Flags indicating errors which have occurred: Bit 0: Laser power fault Bit 1: Laser Quality fault Bit 2: Laser Time fault Bit 3: Laser Hardware Error fault Bit 4: Laser Comms fault Bit 8: Wi-Fi Comms fault Bit 9: Server Details Invalid Bit 10: Wi-Fi Connection Failed Bit 11: TCP Connection Failed Bit 12: Status Upload Failed Bit 13: Sample Upload Failed Bit 14: Config Download Failed (future) Bit 16: Storage Full Bit 17: Flash IC unrecognised Bit 18: Flash IC not responding Bit 19: Flash Erase/Program Error
Battery Voltage	0 ~ 5500 mV
Battery Status	0: Batt Good 1: Batt Low 2: Batt Critical
Current Time	Time in Linux Epoch Format 1/1/1970 ~ 19/1/2038 (s)
Last Distance Measurement	0 ~ 65535 mm
Last Gravity XYZ	3x 10-bit (+/- 2G Scaled)
Temperature	-128 ~ 127°C
Last Sensor Quality	0 ~ 65535

3.4.2 Button Characteristic (UUID: 0032)

This characteristic holds the current state of the button. Enable notifications and short-press the button to verify the button works.

Note that the standard functions will still operate while testing the button. Ie, releasing the button after holding it for more than 3s will turn the Cyclops off, holding the button for more than 5s will reset the Cyclops, and holding the button for more than ~13s will factory reset the Cyclops.

3.4.3 Command Characteristic (UUID: 0033)

This characteristic is used to send service and test commands to the Cyclops. The following commands are supported:

Command	Description	Parameter Meaning
0	Do nothing, successfully	Not Required
1	Trigger Manual Distance measurement and return result	Not Required
2	Trigger Manual Orientation measurement and return result	Not Required
3	Trigger a standard measurement and log result	Not Required
4	Enable laser for targeting	Time (in ms) to enable laser
5	Reserved for future use	Not Required
6	Connect to configured Wi-Fi AP	Not Required
7	Connect to configured server	Not Required
8	Trigger a Wi-Fi upload cycle	Not Required
9	Flash LED	LED Sequence to display: 0: 4x Blue Spin 1: 3x Green Pulse
10	Disconnect from Bluetooth	Not Required
11	Set current time	Current time in Linux Epoch format
254	Enter Shipping Mode	Not Required
255	Factory Reset	Not Required

When a command completes, the result of the command is written to this characteristic and sent as a notification. In general, result codes which are 0 or positive are successful, and negative values indicate errors. The result codes have the following meanings

Value	Description
1	Success, with additional data (see below)
0	Success
-1	General Failure
-2	Cyclops busy handling a previous command
-3	Invalid parameter provided
-4	Invalid command

Wi-Fi error result codes:

Value	Description
-10	Server Details Invalid
-11	Wi-Fi powerup failure
-12	Wi-Fi connection failure
-13	TCP connection failure
-14	Server test endpoint connection failure

Laser error result codes

Value	Description
-20	Laser power low
-21	Laser internal error
-22	Laser temperature too low
-23	Laser temperature too high
-24	Laser out of range
-25	Invalid result
-26	Light too strong
-27	Laser too weak
-28	Laser too strong
-29	Laser hardware error
-30	Laser invalid command
-31	Laser comms error (checksum failure)
-32	Laser comms error (timeout)

The following commands return additional data, as well as the response code.

3.4.3.1 *Manual Laser Distance Measurement*

If the Manual Laser Measurement (Command #1) is successful, the following additional data is returned:

Description	*Type*	*Notes*
Distance	uint32	The distance measured
Quality	uint16	The quality value returned by the laser
Duration	uint16	Duration of the laser measurement, in ms
Error	uint16	Unused

3.4.3.2 *Manual Orientation Measurement*

If the Manual Orientation Measurement (Command #2) is successful, the additional data returned is a single 32-bit value, containing the following data:

Bits	Type	Description
0 ~ 9	int10	Gravity X, +/- 2G
10 ~ 19	int10	Gravity Y, +/- 2G
20 ~ 29	int10	Gravity Z, +/- 2G
30 ~ 31	N/A	Unused

3.5 Device Information Service (UUID: 180A)

The Device Information Service (DIS) consists of the following characteristics:

3.5.1 Manufacturer Name String Characteristic (UUID: 2A29)

This characteristic holds the string “Sandvik”

3.5.2 Model Number String Characteristic (UUID: 2A24)

This characteristic holds the string “01.00”

3.5.3 Firmware Revision String Characteristic (UUID: 2A26)

This characteristic holds the version of the firmware running on the Cyclops, for example, “0.9.0-11”.

3.5.4 Software Revision String Characteristic (UUID: 2A28)

This characteristic holds the string “0”

3.5.5 Hardware Revision String Characteristic (UUID: 2A27)

This characteristic holds the string “00.00”

3.5.6 Serial Number String Characteristic (UUID: 2A25)

This characteristic holds the serial number of the device, eg “000004453”.

3.6 Generic Access Service (UUID: 1800)

This service consists of the following characteristics, required for compatibility with Apple Devices:

3.6.1 Device Name Characteristic (UUID: 2A00)

This characteristic holds the string “TCSS Sensor”

3.6.2 Appearance Characteristic (UUID: 2A01)

This characteristic holds the value 0 (“Unknown Appearance”).

3.7 Silicon Labs OTA Service (UUID: 1D14D6EE-FD63-4FA1-BFA4-8F47B42119F0)

The standard Silicon Labs Secure OTA service is available for over-the-air firmware updates.

4 Hydra

Configuration Service — UUID: bdee0001-7e07-4e68-85f1-d4d87a014940

Set Configuration (Size: 8+ bytes)			Get Configuration (Size: 2 bytes)			Configuration (Size: 8+ bytes)		
Properties: W		UUID 0011	Properties: W		UUID 0012	Properties: R/Notify		UUID 0013
Type	Attribute	Possible Values	Type	Attribute	Possible Values	Type	Attribute	Possible Values
uint16	Configuration Item Id to Set	0 ~ 255	uint16	Configuration Item Id to Get	0 ~ 255	uint16	Configuration Item Id	0 ~ 255
uint8	Length of Value	0 ~ 200				uint8	Length of Value	0 ~ 200
uint32	Update time	1/1/1970 ~ 7/2/2038 (s) (or 0 for now)				uint32	Last Update Time	1/1/1970 ~ 19/1/2038 (s)
byte[0~200]	Configuration Value					byte[0~200]	Configuration Value	

Measurement Service — UUID: bdee0002-7e07-4e68-85f1-d4d87a014940

Retrieve Measurements (Size: 2+ bytes)			Measurement Meta (Size: 4 bytes)			Measurement (Size: 0-MTU)		
Properties: W		UUID 0021	Properties: R/Notify		UUID 0022	Properties: Notify		UUID 0023
Type	Attribute	Possible Values	Type	Attribute	Possible Values	Type	Attribute	Possible Values
uint8	Command	4-5	uint32	Number of Measurements		Custom protocol, see documentation below for more info		
uint8	Number of Samples	0 ~ 16	uint32	Number of Non-Uploaded Measurements				
uint32[]	Samples to Acknowledge		uint32	Number of Measurement Sectors Free				
			uint32	Total Number of Measurement Sectors				
			uint32	Total Measurement data bytes				
			uint32	Non-Uploaded Measurement data bytes				

Status Service — UUID: bdee0003-7e07-4e68-85f1-d4d87a014940

Device Status (Size: 12 bytes)			Unused (Size: 0)			Command (Size: 6+ bytes)		
Properties: R		UUID 0031	Properties: R/W/Notify		UUID 0032	Properties: W/Notify		UUID 0033
Type	Attribute	Possible Values	Type	Attribute	Possible Values	Type	Attribute	Possible Values
uint8	Status	0 ~ 2				uint8	Command	0 ~ 11, 254, 255
uint32	Faults	Flags				uint32	Parameter	
uint16	Battery Voltage	0 ~ 5500 mV				int8	Result Code	
uint8	Battery Status	0 ~ 2				uint8	Result Length	0 ~ 240
uint32	Current Timestamp	1/1/1970 ~ 19/1/2038 (s)				uint8[]	Result Data	

Device Info UUID: 180A

Manufacturer Name String (Size: 7 bytes)		
Properties: R		UUID: 2A29
Type	Attribute	Possible Values
utf8	Manufacturer Name	Sandvik

Model Number String (Size: 5 bytes)		
Properties: R		UUID: 2A24
Type	Attribute	Possible Values
utf8	Model Number	XX.XX

Firmware Revision String (Size: 5 bytes)		
Properties: R		UUID: 2A26
Type	Attribute	Possible Values
utf8	Firmware Revision	XX.XX

Software Revision String (Size: 5 bytes)		
Properties: R		UUID: 2A28
Type	Attribute	Possible Values
utf8	Software Revision	XX.XX

Hardware Revision String (Size: 5 bytes)		
Properties: R		UUID: 2A27
Type	Attribute	Possible Values
utf8	Hardware Revision	XX.XX

Serial Number String (Size: 9 bytes)		
Properties: R		UUID: 2A25
Type	Attribute	Possible Values
utf8	Serial Number	000000001 ~ 999999999

Generic Access UUID: 1800

Device Name (Size: Up to 11 bytes)		
Properties: R/W		UUID: 2A00
Type	Attribute	Possible Values
utf8	Device Name	

Appearance (Size: 2 byte)		
Properties: R		UUID: 2A01
Type	Attribute	Possible Values
uint16	Appearance	0

Configuration Service (UUID: 0001)

The Configuration Service allows getting and setting of configurations in the Hydra. The following Configuration Items are available:

ID	Type	Description	Default
1	uint8	BLE Upload Enable – 0: Disabled, otherwise Enabled	enabled
2	uint8	Wi-Fi Upload Enable – 0: Disabled, otherwise Enabled	enabled
3	String	Wi-Fi SSID – max 31 characters	wifi
4*	String	Wi-Fi PSK – max 31 characters	wifiwifi
6	uint16	Wi-Fi Upload Period – in minutes (0-65535)	60 minutes
7+	String	Server URL – max 47 characters	
8*+	String	API Key – max 47 characters	
9	uint32	Wavelength Priority Threshold – in units of 0.1 picometer (0-2147483647)	5000 (0.5 nanometers)
13	uint16	Status LED Flash Interval in minutes (0-65535)	0 (disabled)
14	uint16	Advertising Interval – in ms (0-65535)	500ms
16	uint8	Wi-Fi network key management protocol [0 = None 1 = WEP, 2 = WPA, 3 = WPA2, 4 = EAP]	3
17	uint8	WPA-EAP Protocol [0= TLS, 1 = PEAP, 2 = TTLS]	0
18*	String	Wi-Fi EAP Anonymous ID – max 31 characters	
19*	String	Wi-Fi EAP ID – max 31 characters	
20	String	Wi-Fi NTP Address – max 43 characters	pool.ntp.org
21	uint8	Wi-Fi NTP Enabled – 0: Disabled, Otherwise Enabled	enabled
23	uint8	Sampling Mode [0 = Config 1 = Static, 2 = Dynamic]	0
24	uint32	Dynamic Mode Pre-Capture Count (0-24000)	500
25	uint32	Dynamic Mode Post-Capture Count (1-24000)	1000
26	uint32	Dynamic Mode Sample Frequency – in units of millihertz	100000 (100hz)
27	uint32	Static Measurement Period – in seconds (10-604800)	15 minutes
28#	String	Interrogator 1 init commands (0-1200 chars). Commands should be separated by a single newline character '\n'	DauSe,0> OBB,0> AO,100>
29#	String	Interrogator 2 init commands (0-1200 chars). Commands should be separated by a single newline character '\n'	DauSe,0> OBB,0> AO,100>
30#	String	Interrogator 3 init commands (0-1200 chars). Commands should be separated by a single newline character '\n'	DauSe,0> OBB,0> AO,100>
31	uint16	Interrogator warmup delay - in seconds (5-300)	30
32	uint8	Overwrite acknowledged measurements when storage is full– 0: Disabled, otherwise Enabled	
129+	uint32	Serial Number (0-4294967295)	0

130	String	BLE advertising name - max 13 characters (set first character as space (0x20) for default value)	SVK + serial number
209*#	uint8[]	AtPKI encoded binary blob containing RADIUS Server CA Certificate(s) for WPA-EAP	
210*#	uint8[]	AtPKI encoded binary blob containing Client Private key(s) for WPA-EAP	
211*#	uint8[]	AtPKI encoded binary blob containing Client Certificate(s) for WPA-EAP	

* These items are write-only and cannot be read.

+ These items are not reset during a Factory Reset

Items are considered "Long Configuration" values and must be read and written using the "Long Configuration" protocol, described in section 2.4.

Note: AtPKI is a container for multiple certificate and key files. Its format is described here

The Configuration Service consists of the following characteristics:

4.1.1 Set Configuration Characteristic (UUID: 0011)

Write the configuration item id, the length of the value, the configuration timestamp, and the new value to the characteristic. The data is validated and applied immediately.

For data to be accepted,

- The provided configuration id must exist
- The provided length must match the provided data and the expected length
- The provided value must be within the permitted values
- The maximum length of configuration value is 200 bytes
- The provided timestamp must be later than the timestamp of the current value, or 0 (which signifies ‘now’, and will be replaced with the current timestamp)

If the new values are acceptable, a notification is sent to the Configuration Characteristic (UUID: 0013). If a write-only or Long-Configuration value is updated, a notification is still sent, but the value is not included in the notification.

The configuration values are stored to flash once the Bluetooth connection is disconnected.

4.1.2 Get Configuration Characteristic (UUID: 0012)

Write the desired configuration id to the characteristic. If the id exists, the current value of the configuration will be sent as a notification to the Configuration Characteristic (UUID: 0013). If a write-only value is requested, a notification is still sent, but the value is not included in the notification.

For Configuration Items set to their default value, the Last Updated Timestamp will be 0.

4.1.3 Configuration Characteristic (UUID:0013)

This characteristic holds the value of the configuration item most recently requested or updated.

4.1.4 Wi-Fi configuration

4.2 Wi-Fi configuration

The Hydra uses ESP-AT firmware on a ESP32-C6 module to connect to wireless networks.

Once the Wi-Fi configuration has been changed, the new configuration will be updated the next time the Hydra periodically uploads data. The 'Connect to configured Wi-Fi AP' command can be used to verify the network configuration is valid.

The following subsections describe how to configure various types of supported Wi-Fi networks.

4.2.1 Unencrypted Personal Networks

Set the following configuration values:

- `WifiSSID` (**3**) to your SSID
- `WifiKeyMgmtProto` (**16**) to 0 (None)

4.2.2 Encrypted Personal Networks

Set the following configuration values:

- `WifiSSID` (**3**) to your SSID
- `WifiPassword` (**4**) set to your Wi-Fi networks PSK
- `WifiKeyMgmtProto` (**16**) to one of 1 (WEP), 2 (WPA), or 3 (WPA2)

4.2.3 Enterprise Networks - PEAP/MSCHAPV2

Set the following configuration values:

- `WifiSSID` (**3**) to your SSID
- `WifiPassword` (**4**) to the RADIUS server password
- `WifiKeyMgmtProto` (**16**) to 4 (WPA-EAP)
- `WifiEapProto` (**17**) to 1 (PEAP)
- `WifiEapAnonIdentity` (**18**) to the RADIUS server anonymous identity
- `WifiEapIdentity` (**19**) to the RADIUS server identity

Optionally the `WifiEapServerCAFile` (**209**) config value can be set to contain the RADIUS serv CA certificate to authenticate the server. The certificate should be encoded using the AtPKI file format and have its 'item type' = 'ca' and PKI index =0.

4.2.4 Enterprise Networks - TLS

Set the following configuration values:

- `WifiSSID` (**3**) to your SSID
- `WifiKeyMgmtProto` (**16**) to 4 (WPA-EAP)
- `WifiEapProto` (**17**) to 0 (TLS)
- `WifiEapAnonIdentity` (**18**) to the RADIUS server anonymous identity
- `WifiEapIdentity` (**19**) to the RADIUS server identity
- `WifiEapServerCAFile` (**209**) to an AtPKI file containing the server Certification Authority (CA) certificate. ('item type' = 'ca')
- `WifiEapClientPrivKeyFile` (**210**) to an AtPKI file containing the client private key ('item type' = 'key')
- `WifiEapClientCertFile` (**211**) to an AtPKI file containing the client certificate file ('item type' = 'cert')

The private key must be unencrypted for the connection to succeed.

4.2.5 Enterprise Networks - TTLS

Set the following configuration values:

- `WifiSSID` (**3**) to your SSID
- `WifiPassword` (**4**) to the RADIUS server password
- `WifiKeyMgmtProto` (**16**) to 4 (WPA-EAP)
- `WifiEapProto` (**17**) to 2 (TTLS)
- `WifiEapAnonIdentity` (**18**) to the RADIUS server anonymous identity
- `WifiEapIdentity` (**19**) to the RADIUS server identity
- `WifiEapServerCAFile` (**209**) to an AtPKI file containing the server Certification Authority (CA) certificate. ('item type' = 'ca')

4.3 Measurement Service (UUID: 0002)

The Measurement Service allows retrieving and acking of sets of measurements taken by the Hydra.

The Measurement Service consists of the following characteristics:

4.3.1 Retrieve Measurements Characteristic (UUID: 00021)

Write commands to this characteristic to retrieve or acknowledge measurements from the Hydra.

The commands available are:

Command	Description	Required Fields
0	[Not used by Hydra, ignored]	Command, Number of Packets. Packet Ids optional.
1	[Not used by Hydra, ignored]	Command, Number of Packets. Packet Ids optional.
2	[Not used by Hydra, ignored]	Command, Number of Packets, and Packet Ids.
3	[Not used by Hydra, ignored]	Command, Number of Packets, and Packet Ids.
4	Optionally ack listed packets, then Send **New** Samples (Hydra Format)	Command, Number of Packets. Packet Ids optional.
5	Optionally ack listed packets, then Send **All** Samples (Hydra Format)	Command, Number of Packets. Packet Ids optional.

These commands are optimised for 2 main scenarios:

4.3.1.1 Professor Download

This scenario requires efficient retrieval of new samples only.

In this scenario, write command 4, and the number of packets to send. Up to that number of samples will be sent to the Measurement Characteristic (UUID: 0023). To Acknowledge the samples, decode each sample received and retain each sample ID send command 4, the number of new samples to send, and the uint32 IDs of the samples decoded previously.

This process can be repeated until there are no new samples transferred in response to the command, this is signified by the Hydra sending a notification of 0 length. At this point, all samples stored in the Hydra are acked.

The number of packets which can be requested is limited while using this command. A maximum of 16 samples can be pending acknowledgement at any time – requesting more than this will return empty notifications until one or more pending samples are acknowledged.

The notification message format is described in section 4.3.3.

4.3.1.2 App Local History

This scenario involves a user requesting the history of a specific Hydra using an app via a local Bluetooth connection.

In this scenario, write command 5, and the number of samples to send. The samples will be sent as a series of notifications to the Measurement Characteristic (UUID: 0023). This can be repeated as required.

The packets sent in this scenario will not be able to be acknowledged, however, if previous packets are pending acknowledgment, providing their packet ids with this command will ack them.

In both scenarios, oldest samples are sent first, and as samples are sent, internal indexes keep track of which samples have been uploaded, and which have not. Once the most recent sample is uploaded, no further samples will be sent. Disconnecting from the Hydra will reset the indexes and allow the same samples to be sent again.

The notification message format is described in section 4.3.3.

4.3.2 Measurement Meta Characteristic (UUID: 0022)

This characteristic holds the current number of acknowledged and unacknowledged samples, as well as the total number of sectors available for samples and unused sector count. This data can be read, or when the count changes, a notification will be sent.

During periods where this value changes rapidly, such as during acknowledgment of many packets, notifications will be paused until the value has not changed for 500ms.

4.3.3 Measurement Characteristic (UUID: 0023)

This characteristic is used to send notifications containing measurements as requested using the Retrieve Measurements Characteristic (UUID: 0021). Subscribe to this characteristic before sending commands to ensure no data is missed.

The potential size of a Hydra measurement is dependent on how many fibres are enabled and the number of channels each fibre is configured with (from a few bytes to ~1 kB). Given this wide range of sizes and the need to chunk larger messages, the measurements to be sent are concatenated and then fragmented into chunks the size of the BLE GATT payload MTU. This means there may be multiple measurements per notification and/or measurements are broken across notifications.

Each measurement is encoded in the method detailed in Appendix A.

Status Service (UUID: 0003)

The Status Service consists of the following characteristics

4.3.4 Button Characteristic (UUID: 0032)

This characteristic is not implemented on the Hydra

4.3.5 Device Status Characteristic (UUID:0031)

This characteristic holds the status of the Hydra. It consists of the following values:

Attribute	Description
Status	0: Good 1: Non-Fatal Error(s) Present 2: Fatal Errors Present
Errors	Flags indicating errors which have occurred: Bit 0: Sensor driver internal fault Bit 4: Sensor Comms fault

	Bit 8: Wi-Fi Comms fault Bit 9: Server Details Invalid Bit 10: Wi-Fi Connection Failed Bit 11: TCP Connection Failed Bit 12: Status Upload Failed Bit 13: Sample Upload Failed Bit 14: Config Download Failed (future) Bit 16: Storage Full Bit 17: Flash IC unrecognised Bit 18: Flash IC not responding Bit 19: Flash Erase/Program Error Bit 21: PSRAM IC not responding
Battery Voltage	0 ~ 5500 mV
Battery Status	0: Batt Good 1: Batt Low 2: Batt Critical 5: Using Ext Power
Current Time	Time in Linux Epoch Format 1/1/1970 ~ 19/1/2038 (s)

4.3.6 Command Characteristic (UUID: 0033)

This characteristic is used to send service and test commands to the Hydra. The following commands are supported:

Command	Description	Parameter Meaning
0	Do nothing, successfully	Not Required
1	Trigger dynamic Mode capture	Not Required
2	Notify next dynamic mode measurement via Measurement Characteristic (UUID:0023)	Not Required
5	Reserved for future use	Not Required
6	Connect to configured Wi-Fi AP	Not Required
7	Connect to configured server	Not Required
8	Trigger a Wi-Fi upload cycle	Not Required
9	Flash LED	LED Sequence to display: 0: 4x Blue Spin 1: 3x Green Pulse
10	Disconnect from Bluetooth	Not Required
11	Set current time	Current time in Linux Epoch format
12	Set current time from configured NTP server	Not required

13	Get Wi-Fi MAC address (returned in Little Endian)	Not required
253*	Clear Sample Non-Volatile Memory	Not required
254	Enter Shipping Mode	Not Required
255	Factory Reset	Not Required

* Note: when clearing Sample memory, a result notification is sent immediately, then sampling is paused by entering Config mode and the LEDs will turn cyan for the duration of the command, which may take up to 3 minutes to complete.

When a command completes, the result of the command is written to this characteristic and sent as a notification. In general, result codes which are 0 or positive are successful, and negative values indicate errors. The result codes have the following meanings

Value	Description
1	Success, with additional data (see below)
0	Success
-1	General Failure
-2	Hydra busy handling a previous command
-3	Invalid parameter provided
-4	Invalid command

Wi-Fi error result codes:

Value	Description
-10	Server Details Invalid
-11	Wi-Fi powerup failure
-12	Wi-Fi connection failure
-13	TCP connection failure
-14	Server test endpoint connection failure

4.4 Device Information Service (UUID: 180A)

The Device Information Service (DIS) consists of the following characteristics:

4.4.1 Manufacturer Name String Characteristic (UUID: 2A29)

This characteristic holds the string “Sandvik”

4.4.2 Model Number String Characteristic (UUID: 2A24)

This characteristic holds the string “03.00”

4.4.3 Firmware Revision String Characteristic (UUID: 2A26)

This characteristic holds the version of the firmware running on the Hydra, for example, “0.9.0”.

4.4.4 Software Revision String Characteristic (UUID: 2A28)

This characteristic holds the string “”

4.4.5 Hardware Revision String Characteristic (UUID: 2A27)

This characteristic holds the string “1.00”

4.4.6 Serial Number String Characteristic (UUID: 2A25)

This characteristic holds the serial number of the device, eg “000004453”.

4.5 Generic Access Service (UUID: 1800)

This service consists of the following characteristics, required for compatibility with Apple Devices:

4.5.1 Device Name Characteristic (UUID: 2A00)

This characteristic holds the string “Sandvik OSD”

4.5.2 Appearance Characteristic (UUID: 2A01)

This characteristic holds the value 0 (“Unknown Appearance”).

5 Professor

Configuration Service — UUID: bdee0101-7e07-4e68-85f1-d4d87a014940

Set Configuration (Size: 8+ bytes)		
Properties: W		UUID 0111
Type	Attribute	Possible Values
uint16	Configuration Item Id to Set	0 ~ 255
uint8	Length of Value	0 ~ 200
uint32	Update time	1/1/1970 ~ 19/1/2038 (s)
byte[0~200]	Configuration Value	

Get Configuration (Size: 2 bytes)		
Properties: W		UUID 0112
Type	Attribute	Possible Values
uint16	Configuration Item Id to Get	0 ~ 255

Configuration (Size: 8+ bytes)		
Properties: Notify		UUID 0113
Type	Attribute	Possible Values
uint16	Configuration Item Id	0 ~ 255
uint8	Length of Value	0 ~ 200
uint32	Last Update Time	1/1/1970 ~ 19/1/2038 (s)
byte[0~200]	Configuration Value	

Status Service — UUID: bdee0102-7e07-4e68-85f1-d4d87a014940

Device Status (Size: 12 bytes)		
Properties: R		UUID 0121
Type	Attribute	Possible Values
uint8	Status	Flags
uint32	Fault Codes	Flags
uint16	Battery Voltage	0 ~ 5500 mV
uint8	Battery Status	0 ~ 4
uint32	Current Timestamp	1/1/1970 ~ 19/1/2038 (s)

Command (Size: 6 bytes)		
Properties: W/Notify		UUID 0122
Type	Attribute	Possible Values
uint8	Command	0 ~ 4, 255
uint32	Parameter/Length	Currently Unused
int8	Result Code	-128 ~ 127

Device Info — UUID: 180A

Manufacturer Name String (Size: 7 bytes)		
Properties: R		UUID: 2A29
Type	Attribute	Possible Values
utf8	Manufacturer Name	Sandvik

Model Number String (Size: 5 bytes)		
Properties: R		UUID: 2A24
Type	Attribute	Possible Values
utf8	Model Number	XX.XX

Firmware Revision String (Size: 5 bytes)		
Properties: R		UUID: 2A26
Type	Attribute	Possible Values
utf8	Firmware Revision	XX.XX

Software Revision String (Size: 5 bytes)		
Properties: R		UUID: 2A28
Type	Attribute	Possible Values
utf8	Software Revision	XX.XX

Hardware Revision String (Size: 5 bytes)		
Properties: R		UUID: 2A27
Type	Attribute	Possible Values
utf8	Hardware Revision	XX.XX

Serial Number String (Size: 9 bytes)		
Properties: R		UUID: 2A27
Type	Attribute	Possible Values
utf8	Serial Number	000000001 ~ 999999999

Generic Access — UUID: 1800

Device Name (Size: Up to 11 bytes)		
Properties: R/W		UUID: 2A00
Type	Attribute	Possible Values
utf8	Device Name	SVKxxxxxxxxxx

Appearance (Size: 2 byte)		
Properties: R		UUID: 2A01
Type	Attribute	Possible Values
uint16	Appearance	0

5.1 Configuration Service (UUID: 0101)

The Configuration Service allows getting and setting of configurations in the Professor. The following Configuration Items are available:

ID	Type	Description	Default	Validation
0^	uint32	Last Update Timestamp	0	-
1+	uint32	Professor Id	0	0-4294967295
2+	String	Professor Name		-
3	String	Professor Group		-
4+*	String	API Key		match regex [A-Za-z0-9_-]*
5	-	Reserved	(Unused)	(Unused)
6+	String	Server URL	https://localhost	Is URL
7	uint32	Maximum FIFO database size - in Megabytes	8192 (8GiB)	5MB - 13 GB
8	uint32	Server Poll Period – in seconds	60	10sec – 60min
9	uint32	FIFO Cache duration – in seconds	60	1sec – 60min
10	uint32	Max elements per FIFO put	100	10 – 500
11	uint32	BLE module Heartbeat period – in seconds	5	1sec – 60sec
12	uint32	Min Shutdown Timeout – in seconds	600	5sec – 10min
13	uint32	Max Shutdown Timeout – in seconds	600	5sec – 10min
14	uint32	Max BLE Connections	2	1 – 4
15	String	Wi-Fi SSID (length of 0 disables Wi-Fi)	wifi	-
16*	String	Wi-Fi network password (either PSK or WPA-EAP identity password depending upon value of WifiKeyMgmt (**17**))	wifiwifi	-
17	uint32	Wi-Fi network key management protocol	3	0='NONE', 1='WEP', 2='WPA', 3='WPA2', 4='WPA-EAP'
18	uint32	WPA-EAP Protocol	1	0='TLS', 1='PEAP', 2='TTLS'
19	String	WPA-EAP anonymous identity		-
20	String	WPA-EAP identity		-
129+	uint32	Serial Number	0	0-4294967295
130^	String	Wi-Fi MAC address		xx:yy:zz:aa:bb:cc
209#	uint8[]	AtPKI encoded binary blob containing RADIUS Server CA Certificate(s) for WPA-EAP		

210*#	uint8[]	AtPKI encoded binary blob containing Client Private key(s) for WPA-EAP	
211#	uint8[]	AtPKI encoded binary blob containing Client Certificate(s) for WPA-EAP	
212#	JSON string	extra arguments for wpa_supplicant in json format e.g {"key":"val","key2":"val2"}	Is Valid JSON

^ Items are read-only and cannot be written.
***** Items are write-only and cannot be read.
+ Items are not reset during a Factory Reset
*Items are considered "Long Configuration" values and must be read and written using the "Long Configuration" protocol in section 2.4.*

Note: AtPKI is a container for multiple certificate and key files. Its format is explained more here.

The Configuration Service consists of the following characteristics:

5.1.1 Set Configuration Characteristic (UUID: 0111)

Write the configuration item id, the length of the value, the configuration timestamp, and the new value to the characteristic. The data is validated and applied immediately.

For data to be accepted,

- The provided configuration id must exist
- The provided length must match the provided data
- The provided timestamp must be later than the timestamp of the current value, or 0 (which signifies 'now', and will be replaced with the current timestamp)

If the new values are acceptable, a notification is sent to the Configuration Characteristic (UUID: 0113). If a write-only value is updated, a notification is still sent, but the value is not included in the notification.

The configuration values are stored to non-volatile memory within 5-10s after the values are modified.

5.1.2 Get Configuration Characteristic (UUID: 0012)

Write the desired configuration id to the characteristic. If the id exists, the current value of the configuration will be sent as a notification to the Configuration Characteristic (UUID: 0113). If a write-only value is requested, a notification is still sent, but the value is not included in the notification.

If all Configuration Items are set to their default value, the Last Updated Timestamp will be 0.

5.1.3 Configuration Characteristic (UUID:0013)

This characteristic holds the value of the configuration item most recently requested or updated.

5.2 Shutdown Timeout

When external power is removed, the Professor is running in 'internal battery' mode, the Professor has two configuration values that influence when the device turns itself off in that mode.

MinShutdownTimeout (**12**) is the minimum amount of time the Professor will wait until it can shut itself down.

MaxShutdownTimeout(**13**) is the maximum amount of time the Professor will wait until it can shut itself down.

In-between the minimum and maximum times, the Professor will decide to shut down if it has met the following criteria:

- If there is no sensor status or message data in its queue
- If it is not connected to any sensors over BLE

5.3 Wi-Fi configuration

The Professor uses wpa_supplicant to connect to wireless networks and so uses similar nomenclature in some of its configuration options.

Once the Wi-Fi configuration has been changed, the new configuration will be updated after the next power cycle, or after the 'Reconfigure Wi-Fi' command is sent.

The following subsections describe how to configure various types of supported Wi-Fi networks.

5.3.1 Unencrypted Personal Networks

Set the following configuration values:

- `WifiSSID` (**15**) to your SSID
- `WifiKeyMgmtProto` (**17**) to 0 (NONE)

Finally send the 'Reconfigure Wi-Fi' command.

5.3.2 Encrypted Personal Networks

Set the following configuration values:

- `WifiSSID` (**15**) to your SSID
- `WifiPassword` (**16**) set to your Wi-Fi networks PSK
- `WifiKeyMgmtProto` (**17**) to one 1 (WEP), 2 (WPA), or 3 (WPA2)

Finally send the 'Reconfigure Wi-Fi' command.

5.3.3 Enterprise Networks - PEAP/MSCHAPV2

Set the following configuration values:

- `WifiSSID` (**15**) to your SSID
- `WifiPassword` (**16**) to the RADIUS server password
- `WifiKeyMgmtProto` (**17**) to 4 (WPA-EAP)
- `WifiEapProto` (**18**) to 1 (PEAP)
- `WifiEapAnonIdentity` (**19**) to the RADIUS server anonymous identity
- `WifiEapIdentity` (**20**) to the RADIUS server identity
- `WifiExtraArgs` (**212**) to ' { "phase1": "peapver=0","phase2": "auth=MSCHAPV2" }'

Optionally the `WifiEapServerCAFile` (**209**) config value can be set to contain the RADIUS serv CA certificate to authenticate the server. The certificate should be encoded using the AtPKI file format and have its 'item type' = 'ca' and PKI index =0.

Finally send the 'Reconfigure Wi-Fi' command.

5.3.4 Enterprise Networks - TLS

Set the following configuration values:

- `WifiSSID` (**15**) to your SSID
- `WifiKeyMgmtProto` (**17**) to 4 (WPA-EAP)
- `WifiEapProto` (**18**) to 0 (TLS)
- `WifiEapAnonIdentity` (**19**) to the RADIUS server anonymous identity
- `WifiEapIdentity` (**20**) to the RADIUS server identity
- `WifiEapServerCAFile` (**209**) to an AtPKI file containing the server Certification Authority (CA) certificate.
- `WifiEapClientPrivKeyFile` (**210**) to an AtPKI file containing the client private key
- `WifiEapClientCertFile` (**211**) to an AtPKI file containing the client certificate file

Optionally if the client private key is encrypted with a password the `WifiExtraArgs` (**212**) must be set to ' {"private_key_passwd": "your_password_here"}'

Finally send the 'Reconfigure Wi-Fi' command.

5.3.5 Enterprise Networks - TTLS

Set the following configuration values:

- `WifiSSID` (**15**) to your SSID
- `WifiPassword` (**16**) to the RADIUS server password
- `WifiKeyMgmtProto` (**17**) to 4 (WPA-EAP)
- `WifiEapProto` (**18**) to 2 (TTLS)
- `WifiEapAnonIdentity` (**19**) to the RADIUS server anonymous identity
- `WifiEapIdentity` (**20**) to the RADIUS server identity
- `WifiEapServerCAFile` (**209**) to an AtPKI file containing the server Certification Authority (CA) certificate.

Finally send the 'Reconfigure Wi-Fi' command.

5.3.6 Supplementary information

Key Files

The table below shows which AtPKI file certificate and key files are mapped to wpa_supplicant configuration variables.

Configuration name	item type	pki index	wpa_supplicant configuration name
`WifiEapServerCAFile`	ca	0	ca_cert
`WifiEapClientCertFile`	cert	0	client_cert
`WifiEapClientPrivKeyFile`	key	0	private_key

Extra wpa_supplicant arguments

Any wpa_supplicant configuration variable can be set inside `WifiExtraArgs` (**22**) and will override previously set values (i.e. "ssid":"test" will override what is set in `WifiSSID`.

The only caveat is that the Professor will automatically surround some configuration values with double quotation marks (") so they do not have to be added (and then escaped) when being entered into JSON strings.

The configuration values below are quoted by the Professor before being passed to wpa_supplicant:

`ssid, psk, wep_key0, phase1, phase2, anonymous_identity, identity, password, private_key_passwd, private_key2_passwd`

As an example, for the following line in a *wpa_supplicant.conf* file

`phase2="auth=MSCHAPV2"`

write the JSON value

```
"phase2": "auth=MSCHAPV2"
```

instead of

```
"phase2": "\"auth=MSCHAPV2\""
```

5.4 Status Service (UUID: 0102)

The Status Service consists of the following characteristics

5.4.1 Device Status Characteristic (UUID:0121)

This characteristic holds the status of the Professor. It consists of the following values:

Attribute	Description
Status	Flags indicating the status of subsystems: Bit 0: Data in FIFO Bit 1: Wi-Fi Network Connected
Errors	Flags indicating errors which have occurred: Bit 0: BLE Communication Channel Down Bit 1: Battery Fault Bit 2: Server Unavailable Bit 3: Server Authentication Failure Bit 4: Professor Configuration Download Error Bit 5: Sensor Measurement/Status Upload Error Bit 6: Professor Status Upload Error Bit 16: System Under Voltage Bit 17: Battery Over Temperature (> 40 deg C) Bit 18: Battery Under Voltage (< 2.85V)
Battery Voltage	0 ~ 5500 mV
Battery Status	0: Batt Good 1: Batt Low 2: Batt Critical 3: Batt Charging 4: Batt Charged
Current Time	Time in Linux Epoch Format 1/1/1970 ~ 19/1/2038 (s)

5.4.2 Command Characteristic (UUID: 0122)

This characteristic is used to send service and test commands to the Sensor. The following commands are supported:

Command	Description	Parameter Meaning
0	Do nothing, successfully	Not Required
1	Flash LED	Not Required
2	Attempt to reconnect to Server	Not Required
3	Test RTC	Not Required

4	Reconfigure Wi-Fi	Not Required
5	Set Time	32-bit Unsigned Unix timestamp
255	Factory Reset	Not Required

When a command completes, the result of the command is sent to this characteristic as a notification. In general, result codes which are 0 or positive are successful, and negative values indicate errors. The result codes have the following meanings

Value	Description
0	Success
-1	Wi-Fi SET_NETWORK_FAILED
-2	Wi-Fi ENABLE_NETWORK_FAILED
-3	Wi-Fi SAVE_CONFIG_FAILED
-4	Invalid parameter
-5	Invalid state
-6	Invalid command
-7	Invalid command length
-8	Generic error

5.5 Device Information Service (UUID: 180A)

The Device Information Service (DIS) consists of the following characteristics:

If any of the values are modified, these characteristics are updated on the next power cycle.

5.5.1 Manufacturer Name String Characteristic (UUID: 2A29)

This characteristic holds the string “Sandvik”

5.5.2 Model Number String Characteristic (UUID: 2A24)

This characteristic holds the string “01.00”

5.5.3 Firmware Revision String Characteristic (UUID: 2A26)

This characteristic holds the version of the firmware running on the Bluetooth Module, for example, “0.1.0-27”.

5.5.4 Software Revision String Characteristic (UUID: 2A28)

This characteristic holds the version of the software running on the Raspberry Pi, for example, “0.2.0”.

5.5.5 Hardware Revision String Characteristic (UUID: 2A27)

This characteristic holds the string “1.00”

5.5.6 Serial Number String Characteristic (UUID: 2A25)

This characteristic holds the serial number of the device, eg “000004453”.

5.6 Generic Access Service (UUID: 1800)

This service consists of the following characteristics, required for compatibility with Apple Devices:

5.6.1 Device Name Characteristic (UUID: 2A00)

This characteristic holds the string “SVKXXXXXXXXX” where XXXXXXXXX is the device serial number. If the device serial number is changed, this characteristic is updated on the next power cycle.

5.6.2 Appearance Characteristic (UUID: 2A01)

This characteristic holds the value 0 (“Unknown Appearance”).

Appendix A. Measurement Encoding scheme

A.1 Hydra Measurement

Each Hydra Measurement is encoded and packetized to allow compact serialization/deserialization and data integrity validation. It is encoded using a transport layer that wraps an application layer payload.

A.2 Transport Layer

The transport layer is used to ensure data integrity and unambiguous packet framing when serialized. This is achieved by using COBS () and a 16-bit CRC.

CRC

This CRC algorithm is CRC-16/CCITT-FALSE

```
Width            : 16
Poly             : 0x1021
Poly(reflected)  : 0x8408
Final XOR        : 0x0000
Initial Value    : 0xFFFF
Standard Check   : 0x29B1
Input reflected  : false
Output reflected : false
```

an example CRC encoding test vector is below (values are hex encoded)

```
input : 31 32 33 34 35 36 37 38 39 ("standard check' vector, ascii '123456789')
output: 0x29B1 (little-endian B1 29 )

input : 01
output: 0xF1D1 (little-endian D1 F1 )

input : 01 02 03 04
output: 0x89C3 (little-endian C3 89 )
```

COBS

Constant Overhead Byte Stuffing (COBS) is used to encode the packets to remove 0x00 bytes from the payload. This value is then used as a delimiter between packets, allowing for arbitrary-length packets to be sent and received, and allowing resynchronization of the decoder if the channel suffers temporary corruption or data loss. Its main advantage is that it guarantees no more than one byte of overhead in every 254 bytes of payload.

To encode the application data payload, it is first checksummed using `CRC-16/CCITT-FALSE`. The checksum is then appended to the payload in little-endian format. The payload+checksum is then COBS encoded.

example packet encoding test vector is below (values are hex encoded)

```
payload: 01
packet: 04 01 D1 F1 00
```

The broken-down packet is below

cobs hdr	payload	crc	cobs delimiter
04	01	D1 F1	00

```
payload: 07 7B 7D 95 32
packet: 06 07 7B 7D 95 32 00
```

The broken-down output is below

cobs hdr	payload	crc	cobs delimiter
06	07 7B 7D	95 32	00

A.3 Application layer

Each application layer packet is made up of a header and a payload.

> The packet consists of an 8-bit packet type followed by it's corresponding Protocol Buffer (v3) encoded payload.

A.4 Hydra Measurement (Packet Type: 1)

The Hydra Measurement protocol buffer schema is stored in the source code repository (D0019734) at the file path `app\src\protobuf\WavelengthAmplitudeData.proto`.

It is also reproduced below.

```
syntax = "proto3";

// Wavelength/Amplitude channel data
message WaveAmpChan {
    sint32 temperature = 1; // temperature(°C)*100 of the on board temperature
sensor
    uint32 refslope = 2; // current reference compensation value; RefSlope=
slope*1,000,000
    uint32 refoffset = 3; // current reference compensation value; RefOffset=
offset(nm)*10,000

   /*
    * wavelength of detected peaks (nanometers*10,000, 1lsb = 100 femtometre)
    *
    * Compression MODE_1
    * To compress the data on-wire, each wavelength value after the first one in
this array is
    * encoded as the difference between itself and the previous wavelength value.
```

```
    * To decode the wavelength of any value, sum the values of itself and all values
before it.
    * e.g.
    * encoded values are W0, W1_diff, W2_diff, W3_diff
    * decoded values are W0, W1, W2, W3
    * To decode W3 from the array
    * W3 = W0 + W1_diff + W2_diff + W_3diff
    */
    repeated sint32 wavelength = 4 [packed=true]  ;

   /*
    * amplitude of detected spectra peaks (unitless * 10,000)
    *
    * Compression MODE_1
    * To compress the data on-wire, each amplitude value is divided by 10,000,
    * as un-averaged data is modulo 10,000.
    */
    repeated sint32 amplitude = 5 [packed=true] ;
}

message Timestamp {
  int64 seconds = 1;
  int32 nanos = 2;
}

enum SampleCompressionMode {
  RAW = 0; // No compression applied (Lossless)
  MODE_1 = 1; // Compression mode 1 applied : wavelength difference, amplitude
resolution reduced (Lossy)
}
enum SampleType {
  DYNAMIC = 0; // This sample is part of a capture
  STATIC = 1; //  This sample is standalone and is not part of a capture
}


// Wavelength/Amplitude data of all channels (plus metadata)
message WaveAmpSample {
    Timestamp timestamp = 1; // timestamp the sample was collected. Unix epoch at
UTC+0.
    uint64 sensor_id = 2; // ID of the sensor that generated this sample
    uint32 sample_id = 3; // sample ID increments for each sample. Use this to
easily validate samples aren't skipped
    bool isHighPriority = 4; // if sample exceeded high-priority threshold
    SampleCompressionMode compression = 5; // compression mode applied to this
sample
    repeated WaveAmpChan channels = 6 ; // map of channels with their data
    SampleType  sample_type = 7; // which mode the sample was captured for
```

```
    uint32 capture_group = 8; // if sample_type is DYNAMIC, represents the capture
this sample belongs to
}
```


ingenuity
CREATE · BETTER · TOGETHER